Guía de Laboratorio: Seguridad Móvil — OAuth 2.0 con PKCE, Almacenamiento Criptográfico y Step-Up 2FA (TOTP)

Curso: Desarrollo Móvil (React Native)
Institución: Universidad de La Sabana — Facultad de Ingeniería
Sesión: Sesión 14 (Compass: Autenticación Móvil, Tokens y Seguridad de Recursos)
Modalidad: Laboratorio Práctico / Prueba de Concepto Controlada
Duración: 1 sesión de clase (sustentación en vivo) + entrega en repositorio de Git

---

1. Contexto y Propósito de Dominio

En el desarrollo de aplicaciones móviles modernas, la seguridad no se reduce a ocultar vistas o almacenar un indicador booleano de sesión (`isLoggedIn = true`). Las aplicaciones instaladas en teléfonos inteligentes son **clientes públicos** (*Public Clients*): no pueden custodiar un secreto de cliente (`client_secret`) sin exponerlo a ingeniería inversa, y operan en entornos donde el tráfico de red y la memoria del dispositivo son auditables.

El propósito de este laboratorio es implementar una arquitectura de seguridad móvil completa y sin librerías "caja negra":

1. **Flujo de Autorización OAuth 2.0 con PKCE (RFC 7636):** Delegar la identidad a un navegador seguro del sistema operativo, eliminando WebViews embebidas inseguras.
2. **Ciclo de Vida de Tokens y Almacenamiento Seguro:** Separar `access_token` volátil en memoria de `refresh_token` duradero en el enclave de seguridad nativo (iOS Keychain / Android Keystore), con intercepción silenciosa de expiración (Silent Refresh Queue).
3. **Módulo Authenticator Propio (RFC 6238):** Implementar el cálculo matemático de códigos TOTP de 6 dígitos desde cero (HMAC-SHA1 + Truncamiento Dinámico) con capacidad de operar en **Modo Avión** y con paridad total frente a Google Authenticator.
4. **Step-Up Authentication & Anti-Replay:** Proteger rutas críticas de negocio mediante elevación dinámica de privilegios y protección contra ataques de repetición en el backend.

Nota de alcance académico: A diferencia de cursos de infraestructura, este laboratorio está **100% enfocado en el ecosistema móvil**. No se requiere instrumentación pesada (sin Prefect

ni clústeres externos). El backend de apoyo puede desarrollarse en Python (FastAPI) o Node.js (Express) con almacenamiento en memoria o SQLite.

---

2. Arquitectura General del Sistema

Flujo de Ejecución

1. **Fase 1: OAuth 2.0 con PKCE**

    - La app genera `code_verifier` y `code_challenge` (SHA-256).
    - Abre el navegador del sistema (`WebBrowser` / `ASWebAuthenticationSession`) hacia `/oauth/authorize`.
    - El proveedor redirige mediante Deep Link: `myapp://oauth/callback?code=AUTH_CODE&state=...`.
    - La app valida el parámetro `state` anti-CSRF y envía `code` + `code_verifier` a `POST /auth/oauth/token`.
    - El backend responde con `access_token` (vida corta en RAM) y `refresh_token` (almacenado en Keychain / Keystore).

2. **Fase 2: Step-Up Authentication**

    - La app solicita un recurso protegido: `GET /api/boveda`.
    - La API responde `403 Forbidden` (`error: "mfa_required"`, con un `step_up_token`).

3. **Fase 3: Cálculo TOTP Local (Modo Avión)**

    - La app calcula localmente el código TOTP(Secret, UnixTime // 30) según RFC 6238.
    - Envía a `POST /auth/mfa/verify` el `step_up_token` y el código de 6 dígitos.
    - El backend valida el código en su caché Anti-Replay y ventanas temporales (C-1, C, C+1).
    - Retorna un `elevated_token` con el claim `mfa_verified: true`.
    - La app vuelve a consultar `GET /api/boveda` y obtiene los datos sensibles.

---

3. Especificación Técnica por Puntos de Trabajo

Punto 1: Flujo OAuth 2.0 con PKCE desde la App Móvil (RFC 7636)

Las aplicaciones móviles no deben solicitar credenciales directamente mediante campos de usuario y contraseña si delegan identidad; deben utilizar navegadores nativos del sistema.

- **Requerimientos de Implementación:**
    1. **Generador Criptográfico de PKCE:**
        - `code_verifier`: Cadena aleatoria de alta entropía (entre 43 y 128 caracteres, URL-safe: `[A-Z]`, `[a-z]`, `[0-9]`, `-`, `.`, `_`, ~).
        - `code_challenge`: Hash SHA-256 del `code_verifier`, codificado en Base64URL sin relleno (*unpadded*).
    2. **Navegador Seguro:** Usar `WebBrowser.openAuthSessionAsync` (Expo / React Native), `ASWebAuthenticationSession` (SwiftUI / iOS) o `Custom Tabs` (Jetpack Compose / Android). Queda **estrictamente prohibido** el uso de `WebView` embebida estándar.
    3. **Deep Linking y Redirección:** Configurar el *custom URL scheme* de la app (ej: `unisabana-auth://callback`).
    4. **Protección Anti-CSRF:** Generar y verificar un parámetro `state` aleatorio único por solicitud. Si el `state` que regresa en el deep link no coincide con el generado en memoria, la app debe abortar el intercambio.
    5. **Token Exchange:** La app envía al backend el `authorization_code` recibido junto con el `code_verifier`. El backend valida que `SHA256(code_verifier) === code_challenge` antes de entregar las llaves de sesión.

Punto 2: Almacenamiento Seguro Nativo y Pipeline de "Silent Refresh"

El error más común en aplicaciones móviles es guardar credenciales en `AsyncStorage`, `localStorage` o `SharedPreferences` sin cifrar.

- **Estrategia de Separación de Memoria:**
    - `access_token`: Vive únicamente en memoria volátil de la app (React State, Context, ViewModel). Se destruye al matar el proceso. Vida útil: 5 minutos.
    - `refresh_token`: Se persiste en el enclave seguro cifrado por hardware (iOS Keychain, Android Keystore vía `expo-secure-store`). Vida útil: 30 días.
- **Requerimientos de Implementación:**

1. **Almacenamiento Cifrado por Hardware:**
    - En React Native / Expo: Utilizar `expo-secure-store`.
    - En iOS nativo: Utilizar **Keychain Services** con clase `kSecClassGenericPassword`.
    - En Android nativo: Utilizar **EncryptedSharedPreferences** respaldado por Android Keystore.
2. **Interceptor de Red con Cola Silenciosa (Silent Refresh Queue):**
    - Configurar un interceptor HTTP (Axios interceptor o cliente wrapper de `fetch`).
    - Cuando una petición a la API responda con un código `401 Unauthorized`:
        1. Pausar todas las llamadas subsecuentes y encolar las peticiones en vuelo.
        2. Disparar una única petición de renovación: `POST /auth/refresh` con el `refresh_token`.
        3. Si la renovación es exitosa, actualizar el `access_token` en memoria, reintentar la petición original y desbloquear la cola.
        4. Si el `refresh_token` también está vencido o revocado, limpiar el almacenamiento seguro y navegar automáticamente a la pantalla de bienvenida/login.

Punto 3: Módulo Authenticator Propio — TOTP desde Cero en Modo Avión (RFC 6238)

Cada equipo debe construir dentro de su app (o en una pantalla dedicada de utilidades de seguridad) su propio generador de contraseñas de un solo uso por tiempo (TOTP).

- **Regla Inviolable: Prohibido el uso de librerías de alto nivel de TOTP** (como `otplib`, `speakeasy`, etc.). Se permite utilizar utilidades nativas de hashing criptográfico (`expo-crypto`, `CryptoKit`, `javax.crypto.Mac`) y decodificadores de Base32.
- **Algoritmo a Implementar:**
    1. **Ventana de Tiempo (T):**
       `T = floor(UnixTimestamp / 30)`
       El valor T debe formatearse como un entero de 64 bits en formato Big-Endian (buffer de 8 bytes).
    2. **Hash Criptográfico:**
       `H = HMAC-SHA1(ClaveSecretaEnBytes, T)`
    3. **Dynamic Truncation (Truncamiento Dinámico):**
        - Leer el último byte de H: `offset = H[19] & 0x0F`.

    - Extraer 4 bytes consecutivos a partir de `offset`: `bin_code = ((H[offset] & 0x7F) << 24) | ((H[offset+1] & 0xFF) << 16) | ((H[offset+2] & 0xFF) << 8) | (H[offset+3] & 0xFF)`
    - Obtener los 6 dígitos: `TOTP = bin_code % 1000000` (Formatear con ceros a la izquierda para garantizar siempre 6 cifras: `054231`).
- **Ingesta del Secreto:**
    - Opción A: Escaneo de código QR mediante la cámara (`expo-camera` / AVFoundation / CameraX) reconociendo el formato URI `otpauth://totp/UniSabana:estudiante@unisabana.edu.co?secret=JBSWY3DPEHPK3PXP&issuer=UniSabana`.
    - Opción B: Campo de texto con validación para pegar manualmente la semilla Base32 (indispensable para pruebas en emuladores).
- **Experiencia de Usuario (UI/UX Móvil):**
    - Tipografía monoespaciada grande para el código de 6 dígitos.
    - Temporizador gráfico (barra de progreso o indicador circular) que decrece en tiempo real de 30 a 0 segundos y refresca el código automáticamente al expirar.
    - Funcionamiento garantizado 100% offline (sin conectividad a internet / **Modo Avión**).

Punto 4: Step-Up Authentication y Backend Anti-Replay

El backend (desarrollado en Python con FastAPI o Node.js con Express) valida el acceso escalonado sin depender de orquestadores externos.

- **Requerimientos de Backend:**
    1. **Protección de Recurso Crítico (`GET /api/boveda-critica`):**
        - Si la petición llega con un `access_token` estándar (donde el claim `mfa_verified` es `false`), la API devuelve `403 Forbidden` con `error: "mfa_required"` y un `step_up_token` temporal.
    2. **Verificación TOTP (`POST /auth/mfa/verify`):**
        - Recibe el `step_up_token` y el código de 6 dígitos ingresado por el usuario.
        - **Tolerancia a Desfase de Reloj (Clock Skew):** Valida en tres ventanas: C-1 (t - 30s), C (t) y C+1 (t + 30s).
        - **Caché Anti-Replay:** Una vez un código es validado para un usuario, se registra en memoria (con TTL de 90 segundos). Si se vuelve a enviar antes de expirar, responde `409 Conflict: Replay Attack Detected`.

3. **Sesión Elevada:** Si la validación es correcta, devuelve un `elevated_token` con el claim `mfa_verified: true` (expiración corta: 10 minutos), permitiendo el acceso a `GET /api/boveda-critica`.

---

5. Pruebas de Aceptación (Sustentación en Vivo)

Durante la sesión de sustentación en el laboratorio, cada equipo contará con **5 minutos** para demostrar los siguientes 5 escenarios en un dispositivo físico o emulador:

#	Prueba de Aceptación	Procedimiento de Demostración	Resultado Esperado
1	Flujo OAuth + PKCE con Navegador del Sistema	Presionar "Iniciar Sesión". La app abre el navegador nativo (no WebView), completa el login y regresa mediante deep link a la app móvil.	La app recibe el callback sin exponer credenciales y almacena el `refresh_token` en `SecureStore`.
2	Paridad Estándar TOTP	Escanear la misma semilla QR en la app del estudiante y en Google Authenticator oficial.	Ambas aplicaciones muestran exactamente los mismos 6 dígitos al mismo tiempo.
3	Cálculo Offline en Modo Avión	Activar el Modo Avión del teléfono. Esperar a que el contador de 30s llegue a 0 y observar la generación del nuevo código.	El código se actualiza matemáticamente sin errores ni peticiones de red.
4	Step-Up Authentication & Bóveda	Intentar abrir la pantalla "Bóveda Crítica". La app debe atrapar el `403 Forbidden`, abrir el modal de 2FA, ingresar el código TOTP y abrir la bóveda.	Acceso autorizado únicamente tras verificar el segundo factor.
5	Ataque de Repetición (Anti-Replay)	Enviar inmediatamente el mismo código TOTP recién usado dentro de la misma ventana de 30 segundos.	El backend rechaza la solicitud (`409 Conflict`) impidiendo el reuso del código.

---

6. Entregables y Rúbrica de Calificación

Entregables

2. **Repositorio de Git:**
   - `/mobile-app`: Código de la aplicación en React Native (Expo), SwiftUI o Kotlin/Compose.
   - `/backend-auth`: Código del backend ligero (FastAPI / Express).
   - `README.md`: Instrucciones paso a paso para ejecutar el backend, configurar variables de entorno y correr la app.

2. **Video Corto Demostrativo (Opcional como respaldo):** 60 a 90 segundos mostrando la prueba en Modo Avión y la sustentación del flujo PKCE.

Rúbrica de Evaluación (Escala 0.0 – 5.0)

Criterio	**Peso**	**Sobresaliente (4.5 - 5.0)**	**Aceptable (3.0 - 4.4)**	**Insuficiente (0.0 - 2.9)**
Implementación Criptográfica TOTP (RFC 6238)	25%	Calcula TOTP desde cero (HMAC-SHA1 + Truncamiento Dinámico) sin librerías externas de TOTP. Funciona en Modo Avión y tiene 100% paridad con Google Authenticator.	Calcula el código pero depende de librerías de alto nivel, o presenta fallas esporádicas de sincronización temporal.	El código no coincide con Google Authenticator o requiere conexión a internet para calcularse.
Flujo OAuth 2.0 con PKCE & Deep Linking	25%	Genera `code_verifier` y `code_challenge` válidos; usa navegador seguro del sistema (`WebBrowser`), maneja deep links y valida `state` anti-CSRF.	Implementa el flujo pero usa WebViews inseguras o no valida adecuadamente el parámetro `state`.	No implementa PKCE o expone el `client_secret` en el código fuente de la app móvil.
Almacenamiento Seguro & Silent Refresh Queue	20%	`access_token` en memoria; `refresh_token` en `SecureStore` (Keychain/Keystore). Maneja 401 con cola de promesas sin cerrar sesión ni desorientar al usuario.	Guarda tokens en almacenamiento seguro pero no cuenta con cola para resolver peticiones concurrentes en 401.	Guarda tokens en texto plano (`AsyncStorage` / `UserDefaults`) o no renueva la sesión.
Step-Up Authentication & Anti-Replay	15%	La API rechaza acceso directo a recursos sensibles; eleva privilegios tras validar 2FA y bloquea ataques de repetición dentro de la ventana de validez.	Implementa Step-Up pero no cuenta con mecanismo para evitar el reuso de códigos en la misma ventana.	No existe elevación de privilegios o cualquier token permite acceso a la bóveda.
Calidad de UI/UX Móvil & Código Limpio	15%	Interfaz pulida, contador visual regresivo (30s), teclado numérico optimizado, arquitectura modular y tipado estricto (TypeScript/Swift/Kotlin).	Interfaz funcional pero con parpadeos visuales al regenerar el código o manejo de errores poco descriptivo.	Código desorganizado, interfaz rota o bloqueos de UI durante operaciones criptográficas.